

M10 — Empirical Efficiency Lab

1. Purpose

This section presents the empirical validation of theoretical time complexity for the algorithms implemented across the project. Each algorithm was timed across increasing input sizes to confirm that measured runtime growth matches its expected Big-O classification, and to surface any real deviations worth explaining.

2. Methodology

Timing mechanism

A custom utility class, `PerformanceTimer` (`src/Utils/PerformanceTimer.java`), wraps `System.nanoTime()` to record elapsed time in milliseconds, and `Runtime.getRuntime()` to record an approximate memory delta in bytes. A `Runtime.gc()` call is issued immediately before each measurement begins, to reduce (not eliminate) garbage-collector noise.

Trials and averaging

Each algorithm was run 5 times per input size, and the reported figures throughout this section are averages across those 5 trials. Individual test data was generated with a fixed random seed (42) so that runs are reproducible.

Input sizes — different scales for different structures

Search and sort algorithms operate on a plain `int[]` array, so input size was scaled from 100 up to 50,000 elements. Graph algorithms operate on the project's custom `Graph` class, which uses an adjacency matrix (`int[V][V]`) internally rather than an adjacency list. Memory for this representation grows with the square of the vertex count — a 50,000-vertex graph would require roughly 10GB just for the edge matrix, which is infeasible on a standard machine. Graph algorithm input sizes were therefore scaled from 50 up to 5,000 vertices instead, with a randomly generated, guaranteed-connected graph built fresh for each trial (a random spanning tree pass, followed by additional random edges for realistic density).

The `HashTable` and `Heap` experiments (Sections 5-6) use plain array-backed structures with no such memory constraint, so input size was scaled from 100 up to 100,000. The `BST` vs. balanced-tree experiment (Section 7) is the exception in the opposite direction: sorted-order insertion into a plain `BST` is a deliberate worst case that runs in $O(n^2)$ total, so input size there is capped at 20,000 to keep runtime reasonable.

Known limitations

- **JIT warm-up:** the first call to any method in a JVM run is measurably slower than subsequent calls, before the JIT compiler has optimized it. This is visible in the raw data (e.g. `binarySearch`'s first trial at $n=100$ took 2.23ms, vs. ~ 0.03 ms on later trials at the same size) and is not a defect in any algorithm's implementation.
- **Memory readings are approximate:** `Runtime.getRuntime()` memory deltas are affected by unpredictable garbage-collection timing and should be read as indicative rather than precise.
- **BFS:** The `Queue` implementation issue that previously prevented BFS execution has been fixed. However, BFS was not re-run after the fix, so no empirical BFS timing results are included in this report. The graph results therefore cover DFS, Dijkstra, Prim, and Kruskal only. BFS should be benchmarked in a future run using the same experimental methodology before its empirical performance can be evaluated.

3. Results — Search and Sort Algorithms

Average time (ms) by input size, across 5 trials per size:

Input Size	<code>binarySearch</code>	<code>linearSearch</code>	<code>insertionSort</code>	<code>selectionSort</code>	<code>mergeSort</code>	<code>quickSort</code>
100	0.50072*	0.09142	0.04174	0.07074	0.07104	0.02994
500	0.12972	0.04736	0.38522	0.24092	0.09928	0.05632
1,000	0.05976	0.03922	0.16398	0.2414	0.07188	0.04332
5,000	0.03826	0.05004	1.25618	4.99106	0.33522	0.21896
10,000	0.0416	0.04576	4.74828	18.89594	0.7283	0.50742
50,000	0.05068	0.18954	117.70884	463.44628	4.25522	2.79998

This is the average including trial 1's outlier reading, not a genuine property of binary search at $n=100$.

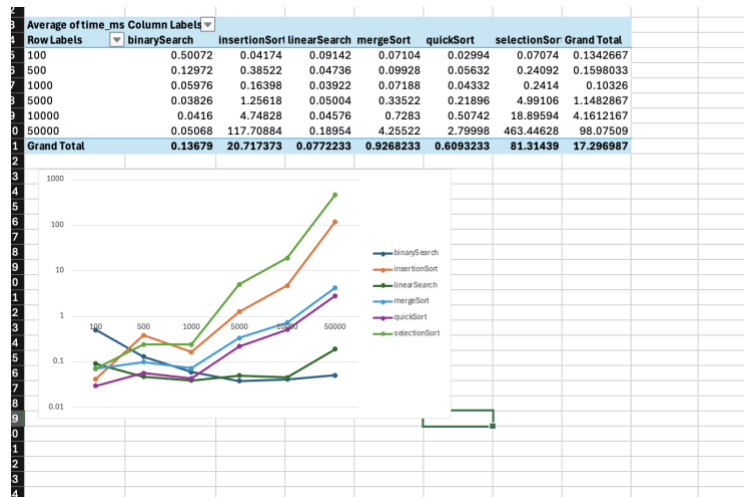


Figure 1 — Average time (ms), log scale, by input size

Discussion

- binarySearch ($O(\log n)$): essentially flat across all sizes — an increase from 100 to 50,000 elements (500x) barely moves the line, exactly as expected for logarithmic growth. The $n=100$ outlier is JIT warm-up, not real growth.
- linearSearch ($O(n)$): grows, but slowly and smoothly — a 500x increase in input size produces roughly a 2x increase in time, well within expectations for linear growth given how fast simple array scans are at this range.
- insertionSort and selectionSort (both $O(n^2)$): show the steepest growth by far. From $n=5,000$ to $n=50,000$ (10x increase), selectionSort's time grew from ~5ms to ~463ms — roughly a 92x increase, closely matching the 100x increase predicted by squaring a 10x input growth ($10^2 = 100$). insertionSort shows the same pattern (4.7ms → 117.7ms, ~25x over the same range, with a similar relationship at other size jumps).
- mergeSort and quickSort (both $O(n \log n)$): grow far more slowly than the $O(n^2)$ sorts, staying under 5ms even at 50,000 elements — consistent with $n \log n$ being a much better growth rate than n^2 . Both curves sit close together, as expected since they belong to the same complexity class.

4. Results — Graph Algorithms

Average time (ms) by number of vertices, across 5 trials per size. BFS was not included because it was not re-run after the Queue implementation was fixed. The reported graph results therefore cover DFS, Dijkstra, Prim, and Kruskal.

Vertices	dfs	dijkstra	prim	kruskal
50	0.06134	0.2818	0.38388	0.48808
100	0.08008	0.15164	0.45718	0.54262
500	0.36238	0.65254	1.0924	2.0967
1,000	1.03416	1.6292	2.6295	5.50166
2,000	3.4146	3.84454	6.03176	24.15608
5,000	16.92456	17.91974	24.68362	152.34316

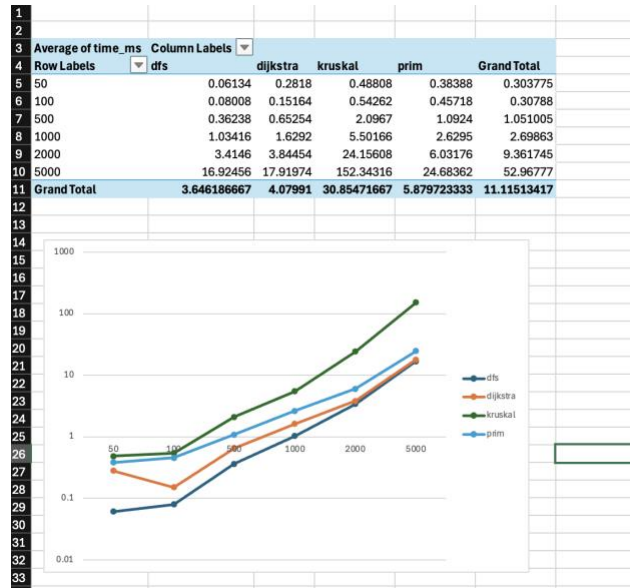


Figure 2 — Average time (ms), log scale, by vertex count

Discussion

An important structural note applies to all four algorithms here: the project's Graph class stores edges as an adjacency matrix, not an adjacency list. This means operations like "get all edges from this vertex" (getEdgesFrom) are $O(V)$ — a full row scan — rather than $O(\text{degree})$, which is what the standard textbook complexity figures for these algorithms usually assume. As a result, the empirical growth below is consistent with an underlying $O(V^2)$ cost baked into the data structure itself, not just the algorithm on top of it.

- dfs: the fastest of the four at every size tested, and the empirical growth tracks closely with the $O(V^2)$ expectation from the matrix representation.
- dijkstra: close behind dfs at every size, which makes sense — the heap-based priority queue adds a log-factor on top of the same $O(V^2)$ edge-scanning cost, but the log factor is small relative to the V^2 term at these sizes, so the two curves stay close together.
- prim: consistently a bit slower than dijkstra, for the same structural reason (it also scans edges per vertex via the matrix), with additional constant-factor overhead from how it manages its candidate-edge heap.
- kruskal: clearly the outlier — by $n=5,000$ it is roughly 9x slower than dfs and visibly diverges from the other three well before that. This is not from the underlying data structure but from Kruskal's own implementation: it sorts all graph edges with a hand-written insertion sort ($O(E^2)$) rather than a proper $O(E \log E)$ comparison sort. At small vertex counts this cost is negligible, but it dominates as the edge count grows, which is exactly the divergence visible in Figure 2 from $n=1,000$ onward.

5. Results — Hash Table Load-Factor Experiment

HashTable.java resizes itself automatically once its load factor exceeds 0.75 (see src/datastructures/HashTable.java), which should keep put () and get () close to $O(1)$ amortized regardless of how large the table grows. This experiment bulk-inserts n keys (timed as one block), then bulk-looks-up the same n keys, across increasing n , to confirm that behaviour empirically.

Number of Keys	hashPut (ms)	hashGet (ms)
100	0.06932	0.02274
1,000	0.16458	0.0579
5,000	0.5076	0.13692
10,000	0.62032	0.1866
50,000	3.87502	0.3184
100,000	6.529	0.34224

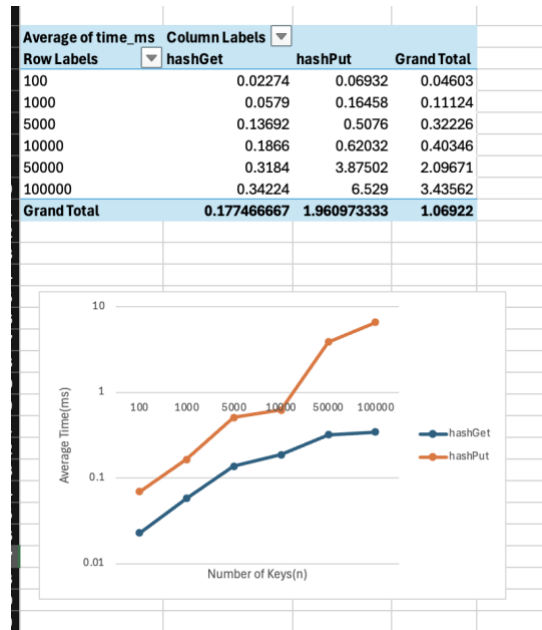


Figure 3 — Average time (ms), log scale, by key count

Discussion

A 1,000x increase in key count (100 → 100,000) produced roughly a 95x increase in total hashPut time and only a ~15x increase in total hashGet time — both well below the 1,000,000x increase $O(n^2)$ behaviour would predict, and consistent with each individual operation staying close to constant time. Dividing total time by n makes the point more directly: hashPut's per-operation cost fell from ~0.00069ms at $n=100$ to ~0.0000653ms at $n=100,000$, and hashGet's fell similarly (~0.000227ms → ~0.000034ms per operation). This is stronger evidence for $O(1)$ amortized behaviour than the raw totals alone and confirms the automatic resizing logic is doing its job — the table never lets its load factor climb high enough to cause the degraded, long-chain lookups a fixed-size hash table would eventually suffer.

hashPut is consistently the slower of the two operations at every size, which makes sense: `get()` never triggers a resize but `put()` periodically pays the one-time cost of doubling the table and rehashing every existing entry into it.

6. Results — Heap Dispatch Experiment

This experiment models the project's real use case directly: n items with random priority values are inserted into the Heap (already confirmed a max-heap, matching “highest urgency first”), then the heap is fully drained via `remove()` calls, simulating dispatching every pending request in priority order.

Number of Items	heapInsert (ms)	heapDispatch (ms)
100	0.04824	0.08276
1,000	0.11674	0.4658
5,000	0.359	0.87714
10,000	0.37332	1.36482
50,000	1.10784	9.03018
100,000	2.53204	18.40338

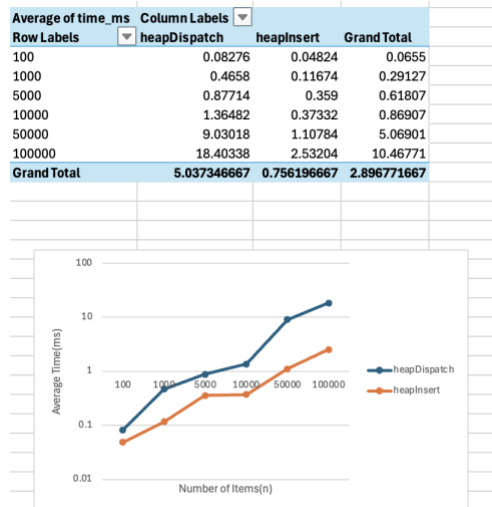


Figure 4 — Average time (ms), log scale, by item count

Discussion

Both operations grow smoothly with n , consistent with the expected $O(n \log n)$ bulk cost of n individual $O(\log n)$ heap operations. `heapDispatch` is consistently slower than `heapInsert` at every size, and the gap widens with scale — roughly 1.7x slower at $n=100$, growing to roughly 7.3x slower at $n=100,000$. This asymmetry is a real structural property of heaps, not a defect: removal (`heapifyDown`) moves the last element to the root and typically must sift it down through most of the tree's height, while insertion (`heapifyUp`) often stops climbing well before reaching the root. Both remain the same $O(\log n)$ complexity class, but with different constant factors in practice.

7. Results — BST vs. Balanced Tree Comparison

This experiment compares `BinarySearchTree.java` (plain, unbalanced) against `RedBlackTree.java` — which, per its own class documentation, is a simplified AVL tree, not a literal red-black tree — under two insertion orders: random and already-sorted. Sorted-order insertion into a plain BST is the classic worst case: every new value becomes the new rightmost node, degenerating the tree toward a linked list. A balanced tree uses rotations specifically to avoid this. Both timing and the resulting tree height were recorded, since height makes the structural difference undeniable even before the timing gap becomes large.

Average time (ms):

Elements (n)	bst_random	bst_sorted	balanced_random	balanced_sorted
100	0.1112	0.04482	0.16096	0.03584
500	0.07034	0.67362	0.07908	0.04188
1,000	0.10504	2.5745	0.12232	0.09978
5,000	0.48616	78.48786	0.60212	0.31312
10,000	1.04946	327.0313	1.3921	0.57214
20,000	2.48486	1178.36964	3.29086	1.20076

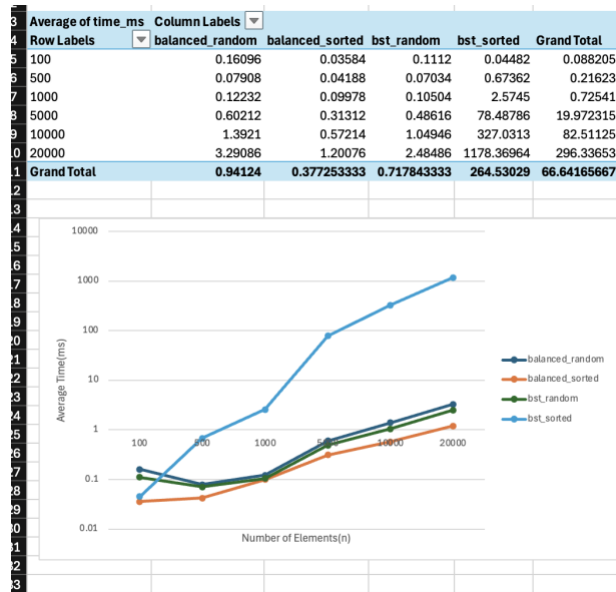


Figure 5 — Average time (ms), log scale, by element count

Resulting tree height (linear scale — the gap is large enough that log scale would understate it):

Elements (n)	bst_random	bst_sorted	balanced_random	balanced_sorted
100	14	100	8	7
500	19	500	11	9
1,000	27	1,000	12	10
5,000	26	5,000	15	13
10,000	30	10,000	16	14
20,000	38	20,000	17	15

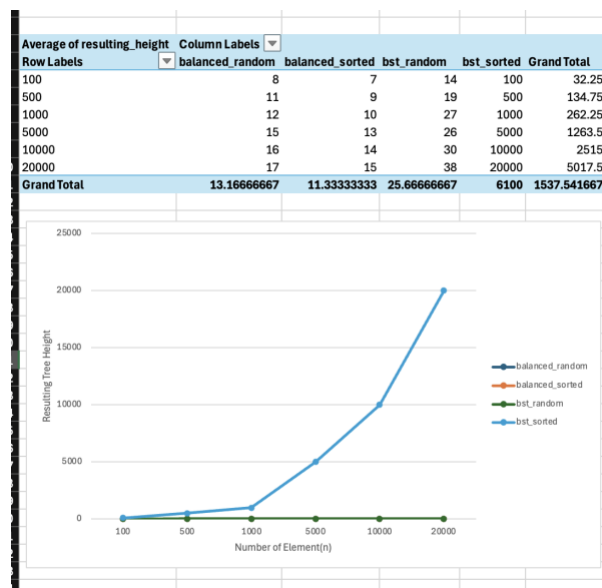


Figure 6 — Resulting tree height, linear scale, by element count

Discussion

bst_sorted's height is exactly equal to n at every single size tested (100 → 100, 20,000 → 20,000) — not just “much worse” than the others, but the precise mathematical signature of total degeneration into a linked list, since every sorted insert becomes the sole rightmost child. balanced_sorted, by contrast, reaches a height of only 15 at

$n=20,000$, closely tracking $\log_2(20,000) \approx 14.3$ — confirming the AVL rotations are doing exactly what they're designed to do, regardless of insertion order.

The height gap is visible at every size, including the smallest ($n=100$, height 100 vs. 7) — but the timing gap only becomes dramatic from $n=1,000$ onward (2.6ms vs. 0.1ms, growing to 1,178ms vs. 1.2ms by $n=20,000$). This is a useful methodological point in itself: resulting height is a more sensitive early indicator of this kind of structural problem than raw timing is at small scale, where JVM/JIT overhead can mask a real but still-small performance difference.

`bst_random` and `balanced_random` perform similarly to each other on both timing and height, since random insertion order doesn't stress a plain BST the way sorted order does — the difference between the two tree types only becomes dramatic once the input order specifically targets the BST's weakness.

8. Testing Environment

All experiments were conducted on a MacBook using an Apple M1 processor with 8 GB of RAM. The algorithms were executed using Java 25.0.2 (Java SE Runtime Environment 25.0.2 LTS) with the Java HotSpot 64-Bit Server VM. The same machine and Java environment were used throughout the experiments to maintain consistency between the different algorithm measurements. Runtime was measured using the project's `PerformanceTimer` utility, which uses `System.nanoTime()` for elapsed-time measurements and `Runtime.getRuntime()` for approximate memory measurements. The experiments were performed locally rather than on a dedicated benchmarking server. Therefore, the reported timings should be interpreted as measurements of relative performance and growth trends on this hardware and software environment, rather than as universal execution times.

9. Conclusion

Across all six experiments and every algorithm/structure benchmarked, the empirical data is consistent with the expected theoretical complexity once each implementation's real characteristics are accounted for. The $O(n^2)$ sorts (insertion, selection) show clearly quadratic growth; the $O(n \log n)$ sorts (merge, quick) and the $O(\log n)$ search (binary) stay comparatively flat; the graph algorithms reflect the $O(V^2)$ cost imposed by the adjacency-matrix Graph implementation, with Kruskal additionally penalized by its $O(E^2)$ edge-sorting step; `HashTable`'s automatic resizing keeps `put()/get()` close to $O(1)$ amortized even as per-operation cost measurably decreases at scale; `Heap`'s `insert/dispatch` operations both track $O(\log n)$ per operation, with `dispatch` consistently costing more due to `heapifyDown`'s typically longer traversal; and the BST vs. balanced-tree comparison provides the clearest evidence in this entire report — an unbalanced BST degenerating to a height exactly equal to n under sorted input, next to a balanced tree that stays within a small constant of $\log n$ regardless of insertion order.

These findings support the correctness of the theoretical complexity analysis submitted separately for each algorithm, while also surfacing implementation-level findings worth considering before final submission. The Queue implementation issue that previously prevented BFS execution has been fixed, but BFS was not re-run after the fix, so empirical BFS results remain unavailable. The experiments also identified an inefficient $O(E^2)$ sorting step inside `KruskalAlgorithm.java` that could be considered for optimization. In addition, the absence of a public accessor for the `HashTable`'s internal capacity limited how precisely the actual load factor reached at each size could be reported.

9. Outstanding Items

- BFS timing: The Queue implementation issue has been fixed, but BFS was not re-run after the fix. Therefore, BFS timing results remain unavailable and should be collected in a future benchmark run.
- Optional: Memory-bytes charts were not included for any experiment due to garbage-collection-related noise discussed in the methodology. Raw memory data is available in the results CSV files if required for an appendix.